A Worked Case: Trust Gates on Pull Request #44844

`Azure/azure-sdk-for-python`, authored by the GitHub Copilot coding agent

Supplement to *Engineering Trust When AI Writes Your Software*

Evidence gathered 2026-09-28/29; all commands reproducible with the replication package

Abstract

This supplement walks through the second worked case of the case study. PR #44844 fixed a real crash (a `RecursionError` when log-capturing contexts nest) and shipped with tests, a changelog entry, and a fluent description. It also changed which run a line of output is attributed to, for a usage pattern its author never considered and no requirement ever stated. The companion supplement on #41852 shows a defect the project later found itself; this one shows a defect nobody has found yet, still present on `main`, and it makes two points the first case cannot: that the violated requirement had to be reconstructed from how the production caller uses the class, and that a claim check is only as good as the call sequence it mimics, since our first attempt at this check was wrong.

1 The pull request as the reviewer saw it

The task. During evaluation runs, `NodeLogManager` redirects `sys.stdout` and `sys.stderr` into per-run buffers so that each evaluated line's output can be stored with its run id. When evaluations nest (an evaluator that itself runs an evaluation), each nested manager wrapped the previous writer, and every `write` recursed through the whole chain. Deep enough nesting raised `RecursionError`. The GitHub Copilot coding agent was assigned the fix.

The description. The PR body: "Nested NodeLogManager contexts could recursively forward stdout/stderr through stacked NodeLogWriter instances, leading to RecursionError during evaluation runs. This change ensures nested logging remains safe and deterministic under concurrent execution." Four bullets: *logging guardrails* ("unwrap chained NodeLogWriter targets with cycle detection and safe fallback to real streams"); *idempotent wrapping* ("reuse existing NodeLogWriter instances when streams are already wrapped"); tests; changelog. An example follows: two nested `with NodeLogManager():` blocks containing a `print`.

The diff. In `_legacy/_common/_logging.py` (+29/−4), the manager's constructor reuses an existing writer instead of wrapping it:

```
 def __init__(self, record_datetime: bool = True):
-    self.stdout_logger = NodeLogWriter(sys.stdout, record_datetime)
-    self.stderr_logger = NodeLogWriter(sys.stderr, record_datetime, is_stderr=True)
+    if isinstance(sys.stdout, NodeLogWriter):
+        self.stdout_logger = sys.stdout
+    else:
+        self.stdout_logger = NodeLogWriter(sys.stdout, record_datetime)
+    if isinstance(sys.stderr, NodeLogWriter):
+        self.stderr_logger = sys.stderr
+    else:
+        self.stderr_logger = NodeLogWriter(sys.stderr, record_datetime, is_stderr=True
    )
```

and the writer gains a fallback stream and a resolver that unwraps chains with cycle detection:

```
+    self._fallback_out = sys.__stderr__ if is_stderr else sys.__stdout__
 ...
-        return self._prev_out.write(s)
+        out = self._resolve_prev_out()
+        return out.write(s) if out is not None else 0
 ...
+def _resolve_prev_out(self):
+    current = self._prev_out
+    visited = set()
+    while isinstance(current, NodeLogWriter):
+        if id(current) in visited:
+            return self._fallback_out
+        visited.add(id(current))
+        current = current._prev_out
+    return current if current is not None else self._fallback_out
```

A new `tests/unittests/test_logging.py` (42 lines) holds two tests, and `CHANGELOG.md` gains "Prevent recursive stdout/stderr forwarding when NodeLogManager is nested, avoiding RecursionError in concurrent evaluation runs."

The merge. Squash commit `7d26b949`, merged 2026-02-03 into `main`. CI green; one human approval. The description's own example (two nested managers, no run contexts) is exactly what the tests exercise, and it works.

2 Requirements and trust hypotheses

2.1 Extracting the requirements

The issue and description state one requirement. The second had to be reconstructed, and how that was done matters.

R1 (stated). Nested log-manager contexts do not recurse.

R2 (unstated). Output written inside a run is attributed to that run's log, including after a nested run has finished. Nothing in the issue, the description or the tests mentions attribution. It surfaces only by asking what the class is *for*: its docstring says "intercepts and saves logs to stdout/stderr when executing a node", and its only production caller, `_batch_engine/_engine.py`, does this for every evaluated line:

```
@contextmanager
def _exec_line_context(self, run_id, line_number):
    log_manager = NodeLogManager()
    log_manager.set_node_context(run_id, "Flex", line_number)
    with log_manager, self._update_operation_context(run_id, line_number):
        yield
```

Every line gets its own run id, and the per-run log is the product the manager exists to make. A requirement that nobody wrote down is still a requirement; the framework's instruction to extract requirements *before* reading the change is what forces the question "what must this fix not break?" to be asked.

R3 (stated, weakly). When the real streams have been replaced, writes fall back somewhere safe.

2.2 The two-question procedure, applied

Q1: would a violation be a security, safety, data-integrity or compliance failure, or make the artifact unfit for purpose? *Q2:* does correct behaviour depend on resolving an ambiguity, on unobservable state, on open-ended completeness, or on behaviour with no reference implementation?

	Q1	Q2	Level	Reasoning
R1	yes	no	MUST-HOLD (H1)	A crash during evaluation runs makes the SDK unfit; the behaviour is deterministic and directly testable.
R2	**yes**	**yes**	MUST-HOLD (H2)	Logs attributed to the wrong run are silently wrong data for the user (Q1). Whether attribution survives the fix depends on an unstated design question, "should nested managers share state?", which the change answers one way without saying so (Q2).
R3	no	yes	CANNOT-GUARANTEE (H3)	What "safe" means when `sys.__stdout__` itself has been replaced (as the tests do) has no reference; the fix can only promise a best effort.

Where the ambiguity is. In #41852 the ambiguity was about how users import the package. Here it is about ownership of state. A `NodeLogWriter` holds three things: the stream it forwards to, a `ContextVar` naming the current run, and a dictionary from run id to buffer. Before the PR, each manager owned its own writer and therefore its own context variable; after it, nested managers share one writer and one context variable. "Idempotent wrapping" sounds like a property of wrapping; it is in fact a decision that the inner run and the outer run share a single notion of "current run". Classifying R2 as must-hold *and* ambiguity-dependent is what obliges the claim check to exercise the shared-state reading rather than the description's example.

2.3 Gate plan

Hypothesis	Level	Gates
H1 no recursion	MUST-HOLD	G2/G3 (is the guard defended?), G4 (cycle check)
H2 attribution preserved across nesting	MUST-HOLD	G4 (with the production caller's sequence); G2/G3 (are the reuse branches defended?)
H3 fallback when real streams are replaced	CANNOT-GUARANTEE	G2/G3 only; recorded as a limitation
H0 referenced interfaces exist (standing, every PR)	MUST-HOLD	G1

> Classification by a single analyst (the AI assistant) who had already seen the diff and the gate results, so it is not blind; the human rater's description-only classification supersedes it (that rater extracted R1 and R3, not R2). R2 is the contentious row: a rater who reads "deterministic under concurrent execution" as covering attribution might still assign must-hold, but would answer Q2 "no" and would then not be prompted to test the shared-state reading. That difference in Q2, not in the level, is what the procedure is designed to surface.

3 Environment and baseline

```
git checkout 7d26b949
export PYTHONPATH=$PWD/sdk/evaluation/azure-ai-evaluation
export AZURE_TEST_RUN_LIVE=true AZURE_SKIP_LIVE_RECORDING=true
python -m pytest -q tests/unittests/test_logging.py    # 2 passed
grep -rl NodeLogManager tests/                          # only test_logging.py
```

The last line matters: the two tests the PR wrote are the only tests anywhere in the package that touch this module. The package suite and the PR's own suite are the same two tests, so G2 and G3 coincide by construction. Baseline green.

4 G1: interface resolution

25 changed lines; every import and every `module.attr` reference on them resolved; `pyflakes` reported no undefined names. Pass, no findings. As in the companion case, a pass rules out one failure mode and nothing more; `not_evaluated` records that calls on objects of inferred type (`current._prev_out`, `out.write`) are outside G1's reach.

5 G2 and G3: mutation on the changed lines

Thirteen single-point mutants on the changed lines; six killed, seven survived, identically for the package suite and the PR's tests.

Line	Code	Mutation	Result
174	`if isinstance(sys.stdout, NodeLogWriter):`	negate condition	**survived**
178	`if isinstance(sys.stderr, NodeLogWriter):`	negate condition	**survived**
181	`is_stderr=True`	`True`→`False`	survived
263	`return out.write(s) if out is not None else 0`	`is not`→`is`	killed
263	same	`0`→`1`	survived
263	same	`return ...`→`return None`	killed
282	`if out is not None:  out.flush()`	`is not`→`is`	survived
282	same	negate condition	survived
294	`if current_id in visited:`	`in`→`not in`	killed
294	same	negate condition	killed
295	`return self._fallback_out` (cycle)	`return None`	survived
298	`return current if current is not None else ...`	`is not`→`is`	killed
298	same	`return None`	killed

Reading the survivors. The resolver's loop (lines 294, 298) is defended: the 500-deep test kills anything that breaks unwrapping. The two constructor branches (174, 178), the ones that implement "idempotent wrapping", are not: negate either and both tests still pass, because with the resolver in place a chain of wrappers unwinds correctly whether or not writers are reused. So the tests establish H1 and say nothing about the reuse decision at all. Lines 181, 263, 282 and 295 are the fallback and flush paths, never reached by any test; that is H3's evidence gap.

What G2/G3 contributed. Not a defect but a direction. The PR's second headline feature has no test that would notice its absence, and it is the feature that changes state ownership. That is where G4 has to look.

6 G4: claim checks

6.1 The claims

(a) Chained writers are unwrapped with cycle detection and safe fallback to real streams.
(b) Existing writers are reused rather than re-wrapped.
(c) Nested logging remains safe and deterministic under concurrent execution.

(d) The tests cover nested writers and nested manager usage.

(a) and (b) are directly checkable. (c) is the claim that carries H2; "deterministic" is a property of what gets logged where, and the description's example, two managers with no run contexts, cannot show it because without contexts nothing is attributed to anything. The check for (c) therefore has to set run contexts the way the production caller does. (d) is settled by Section 5.

6.2 Checks C1 and C2

```
a = NodeLogWriter(io.StringIO()); b = NodeLogWriter(a); a._prev_out = b     # C1: a two
    -writer cycle
b.write("cycle")               # parent: RecursionError;  PR: returns, text reaches sys
    .__stdout__
with NodeLogManager() as outer:
    with NodeLogManager() as inner:
        inner.stdout_logger is outer.stdout_logger      # C2: parent False;  PR True
```

Both claims supported.

6.3 Check S, first attempt (discarded)

The first version of the attribution check was written from a plausible-looking scenario: an outer manager sets context `r1`, an inner manager sets `r2` and then explicitly *clears* it before exiting. At the merge commit, outer output written after the inner block leaked to the real `stdout` instead of `r1`. That looked like a defect, but the scenario was invented: nothing in the code base calls `clear_node_context`. A check that mimics a call sequence the software never performs proves nothing about the software. The check was rewritten from the production caller.

6.4 Check S, from the production caller

`_exec_line_context` constructs the manager, sets the run context, then enters; it never clears. Nesting it once:

```
outer = NodeLogManager(); outer.set_node_context("r1", "Flex", 1)
with outer:
    print("outer-before")
    inner = NodeLogManager(); inner.set_node_context("r2", "Flex", 1)
    with inner:
        print("inner-line")
    print("outer-after")
o, i = outer.get_logs("r1")["stdout"], outer.get_logs("r2")["stdout"]
```

	`outer-after` in r1	`outer-after` in r2	`outer-after` on real stdout
Before the PR (parent)	yes	no	no
After the PR (merge commit)	**no**	**yes**	no

(`inner-line` shows as absent from `r2` at the parent only because the query goes through the outer manager's writer; at the parent the inner writer was a separate object with its own buffers. This is a property of the check's bookkeeping, not a finding.)

6.5 Why

Before the PR: the inner manager wraps the outer writer in a new writer with its own `ContextVar`; `set_node_context("r2")` sets that inner variable; on `__exit__`, `sys.stdout` is restored to the outer writer, whose own variable still says `r1`; `outer-after` lands in `r1`. After the PR: the

inner manager *is* the outer writer; `set_node_context("r2")` overwrites the one shared variable; `__exit__` restores `sys.stdout` to the same object, and nothing restores the variable, because before the PR nothing needed to; `outer-after` lands in `r2`, whose buffer exists in the shared dictionary, so the line is not lost but misfiled. The fix for the recursion is exactly the cause of the misattribution: two properties of "one writer per manager" were doing work, and the PR kept one.

7 Triage

Confirmed defect? Yes: H2 violated with the production caller's exact sequence, reproduced by script, before/after.

Escaped? Yes: CI green; no review comment mentions attribution, contexts or shared state; present in merged code; still present on `main` on 2026-09-28.

Failure mode. Assumption injection: "should nested managers share state?" was answered "yes" implicitly, and the consequence for the run context was not considered.

Severity. Major: silently wrong per-run logs whenever evaluations nest, which is the situation the PR exists to handle.

Second finding. Evidence gap: 7/13 mutants survive, including both reuse branches; H3's paths untested.

Not evaluated. Concurrent execution (claim (c)): `ContextVar` values are per task, so sharing a writer across concurrent tasks may well be safe, but this was not exercised. The `stderr` writer, symmetric code, not separately exercised.

> Single-analyst triage, recorded in `results/evidence/pr44844_G4.json`; independent human re-run and re-judgement required before citation.

8 Downstream status

No downstream confirmation exists. `git show main:.../_logging.py` on 2026-09-28 still contains the reuse branches, and no later PR touches attribution. That is itself a data point: the misattribution produces no exception, no failed test and no visible error, only logs filed under the wrong run, which is the kind of defect the paper's introduction describes as surfacing "only when the system reaches conditions neither the model nor the team explicitly considered". It was reported to the project as issue #49274 on 2026-09-30; a maintainer confirmation would give this case the external anchor that #42313 gives the companion case.

9 The claim–evidence–limitation statement

This is the statement shown as Figure 2 in the article.

Claim	Evidence	Limitations
Nested `NodeLogManager` contexts no longer recurse, and output written inside them reaches the underlying stream. *Scope:* nesting *without* per-run context switches. *Serves:* H1 must-hold, H3 cannot-guarantee.	PR test: 500-deep nesting, no `RecursionError`, text reaches base streams. G4 C1: two-writer cycle resolves to `sys.__stdout__`. G4 C2: inner manager reuses the outer writer. G1: 25 lines, all references resolve. G2/G3: 6 of 13 mutants killed.	Per-run attribution across a nesting boundary is **incorrect** (H2 must-hold, violated): with the batch engine's call sequence, outer-run output after a nested run is logged under the inner run id. Confirmed; on `main`. 7/13 mutants survive, including both reuse branches. Concurrent execution, also claimed, not evaluated. `stderr` path not separately exercised.

The scope says "without per-run context switches"; the first limitation says what happens with them. A reviewer holding this statement next to a diff that adds `isinstance(sys.stdout, NodeLogWriter)` to a constructor would have had the question in front of them.

10 What each gate contributed, and what it cost

Gate	Contribution	Cost
G1	Ruled out hallucinated interfaces	seconds
G2/G3	Showed the reuse branches (the state-ownership decision) are undefended by any test	<1 min machine
G4	Found the defect, after one discarded check; identified the mechanism	≈35 min analyst, including reading the caller
Downstream search	Established that the defect is unfound and still present	≈5 min

The counterfactual. The pipeline reported that the agent's two tests pass, and they test the property the agent was asked to deliver. Static analysis has nothing to say about which object owns a `ContextVar`. The reviewer saw a crash fixed, tests added, a changelog entry, and an example that runs. What found the defect was writing down a requirement the PR never mentioned, classifying it as must-hold and ambiguity-dependent, and then refusing to accept a check that did not mimic the production caller. The first discarded check is part of the method, not an embarrassment to hide: the gate's value depends entirely on the fidelity of what it exercises.

11 Reproduction

```
python run_experiments.py run --prs 44844              # G1, G2/G3, G4
cat evidence/pr44844_G4_output.txt                       # the before/after table of
    Section 6
python run_experiments.py compare --prs 44844
sed -n 443,448p azsdk/sdk/evaluation/azure-ai-evaluation/azure/ai/evaluation/_legacy/
    _batch_engine/_engine.py   # the caller
```

Notes on method

The discarded first check is the methodological lesson of this case: a claim check must be derived from the description and from the software's real call sequences, never from a scenario that merely

seems plausible. Every judgment in Sections **??** and 7 is currently single-analyst; the human classification sheet and an independent re-run of Section **??** are prerequisites for citing this case.